KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

KG COLLEGE OF ARTS AND SCIENCE

Technical Department

JAVA PROJECT REPORT ON

SECURE USER LOGIN AND AUTHENTICATION SYSTEM WITH PASSWORD VALIDATION

Submitted by

ROHITH K

25BIT141

Under the Guidance of

MR. MANEENDHAR

MR.MUGILAN

Technical Department, KG College of Arts and Science

Academic Year: 2025 – 2028

APRIL 2026

Semester: II

DECLARATION

I hereby declare that the project entitled **"Secure User Login and Authentication System with Password Validation"** submitted in partial fulfillment of the requirements for the course **Java Programming** is a record of my original work carried out under the guidance of **Mr. Mugilan**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026
Coimbatore

ROHITH K

CERTIFICATE

This is to certify that the project entitled **"Secure User Login and Authentication System with Password Validation"** is a bona fide work carried out by **ROHITH K B.Sc.IT INFORMATION TECHNOLOGY, KG College of Arts & Science**, in partial fulfillment of the requirements for the course **Java Programming**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: Coimbatore
Date:

Submitted for the Viva-Voce Examination held on _____

Guide:	**Head of the Department**
(Technical Trainer, KGCAS)	(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page No.
1	**Introduction**	6
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	8
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scope and Limitations	
3	**System Design**	10
3.1	Modules	
4	**Methodology**	12
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	**17**
6	**Output Screens**	21
7	**Discussion and Conclusion**	21
7.1	Discussion	
7.2	Conclusion	
8	**References**	**23**

ABSTRACT

The Secure User Login and Authentication System with Password Validation is a Java-based console application designed to implement secure user authentication with strict password format enforcement. In many real-world applications, user login systems are vulnerable to weak passwords and repeated unauthorized access attempts. This project addresses these security concerns by implementing a robust authentication mechanism with password validation and account lockout after multiple failed attempts.

The system accepts a username and password, validates the password format using a dedicated method, and authenticates the user against stored credentials. The application is developed using Core Java programming concepts including methods, boolean logic, character operations, conditional statements, loops, and Scanner for input handling.

The main objective of the system is to enforce strong password policies, prevent brute-force access, and provide clear feedback to users on every login attempt including successful login, invalid format, wrong credentials, and account lockout.

This project demonstrates the practical implementation of Core Java programming concepts and provides a strong foundation for developing more advanced secure authentication systems in the future.

1. INTRODUCTION

User authentication is a fundamental component of any secure software system. It ensures that only authorized users can access protected resources and sensitive data. In many applications, weak or improperly validated passwords allow attackers to gain unauthorized access, leading to data breaches and security vulnerabilities. A robust login system must verify user identity accurately and respond appropriately to invalid or weak password attempts.

With the advancement of cybersecurity awareness, developers are required to build applications that enforce strong password policies and protect against brute-force attacks. A secure login system must validate password format, check credentials against stored data, limit failed attempts, and lock accounts when suspicious activity is detected.

The Secure User Login and Authentication System with Password Validation is a Java-based console application designed to authenticate users with a registered username and password. The system validates password format, checks credentials, limits login attempts to three, and locks the account after three consecutive failures. It provides clear feedback for each login scenario including invalid format, invalid credentials, and account lockout.

This project is developed using Core Java programming concepts including methods, conditional statements, loops, character operations, and Scanner for user input handling.

1.1 Objective

The main objective of the Secure User Login and Authentication System is to develop a secure and reliable Java-based login system that enforces password validation rules, authenticates users against stored credentials, and protects accounts from unauthorized access through attempt limiting and account lockout.

The specific objectives of this project are:

- To design and develop a Java-based secure login system with password format validation.
- To authenticate users by verifying their credentials against stored username and password.
- To limit login attempts to three and lock the account after three consecutive failures.

- To provide clear and specific feedback messages for each login scenario.
- To implement a password validation method that checks for uppercase, lowercase, digit, and special character.
- To demonstrate the use of Core Java concepts including methods, loops, conditionals, and character operations.
- To build a console-based application that handles edge cases such as invalid format before credential checking.

1.2 Overview

The Secure User Login and Authentication System is a console-based application developed using the Java programming language. The system provides a secure login interface where users must enter a valid username and password that meets defined format requirements.

The system stores a predefined username and password. On each login attempt, the password entered is first validated for format compliance, then checked against the stored credentials. The system allows a maximum of three attempts before locking the account permanently for that session.

The application provides real-time feedback on each login attempt, informing the user of invalid password format, wrong credentials, remaining attempts, and account lockout status.

1.3 Features

The Secure User Login and Authentication System includes the following main features:

- Password Validation – Validates password format by checking for minimum 8 characters, uppercase letter, lowercase letter, digit, and special character.
- User Authentication – Checks entered username and password against stored credentials to verify identity.
- Attempt Tracking – Tracks failed login attempts and displays remaining attempts after each failure.
- Account Lockout – Locks the account after three consecutive failed login attempts and prevents further access.

- Informative Feedback – Provides specific messages for invalid password format, wrong credentials, remaining attempts, and account locked status.
- Login Success Message – Displays Login Successful message when the correct username and valid password are entered.
- Console-Based Interface – Operates through the Java console, allowing users to enter credentials and receive instant feedback.
- Secure Design – Implements security-first design by combining format validation with credential checking and lockout protection.

2. PROGRAM ANALYSIS

Program analysis is a critical phase in software development where the problem is studied in detail and the system requirements are identified. This stage helps in understanding how the authentication system should work and what security mechanisms are required to address identified vulnerabilities.

The analysis phase focuses on identifying security gaps in existing login implementations and proposing an authentication solution that enforces password rules, limits attempts, and protects accounts.

2.1 Problem Definition

In many software applications, login systems lack proper password validation and account lockout mechanisms. This creates serious security vulnerabilities that can be exploited by attackers.

Common security problems in existing login systems include:

- Login systems without password validation accept weak passwords that are easy to crack.
- Applications without attempt limits are vulnerable to brute-force attacks.
- Users are not informed clearly when a password does not meet format requirements.
- Systems without account lockout allow unlimited unauthorized login attempts.
- Poor feedback messages confuse users about the reason for login failure.
- No session-level protection is provided when repeated failures occur.

Insecure login systems expose user accounts to unauthorized access, leading to data theft, privacy violations, and system compromise.

Therefore, there is a clear need for an authentication system that enforces password format rules, validates credentials securely, tracks failed attempts, and locks accounts when suspicious behavior is detected.

2.2 Project Overview

The Secure User Login and Authentication System is a Java-based console application developed to demonstrate secure user authentication with password format validation. The system provides a structured login interface with credential checking, attempt tracking, and account lockout.

The system provides the following functionalities:

- Validating password format to ensure it meets minimum security requirements.
- Authenticating users by matching entered credentials with stored username and password.
- Tracking the number of failed login attempts and displaying remaining attempts.
- Locking the account after three consecutive failed login attempts.
- Providing clear feedback messages for each login scenario.
- Handling edge cases such as invalid password format before credential checking.

This project is implemented using Core Java programming concepts including methods, loops, conditionals, character operations, and Scanner for console input.

2.3 Scope and Limitations

Scope

The scope of the Secure User Login and Authentication System includes:

- Validating user password format against defined security rules.
- Authenticating users with stored username and password credentials.
- Limiting login attempts to a maximum of three per session.
- Locking accounts after three consecutive failed attempts.
- Displaying appropriate feedback messages for all login scenarios.
- Supporting clean program termination after successful login or lockout.

The system can be used in:

- Student project login systems
- Small business applications requiring user login
- Internal tools requiring basic access control
- Educational demonstrations of authentication logic

Limitations

- The current version uses hardcoded credentials; a database-backed user store is not implemented.
- Multi-user support and role-based access control are planned for future versions.
- Password hashing and encryption are not implemented in this basic version.
- The lockout does not persist beyond the current program session.

3. SYSTEM DESIGN

System design is the phase where the structure and components of the authentication system are planned before implementation begins. In this phase, the system is divided into modules, and the login workflow is clearly defined. Proper system design ensures the authentication logic is organized, secure, and easy to maintain.

The Secure User Login and Authentication System is designed as a structured application where each component handles a specific aspect of the authentication process.

3.1 Modules

The main modules of the Secure User Login and Authentication System are explained below:

1. Password Validation Module

This module handles password format validation before any authentication is attempted. It checks whether the entered password meets all required security criteria.

Functions of this module:

- Check password length is at least 8 characters.
- Verify the presence of at least one uppercase letter.
- Verify the presence of at least one lowercase letter.
- Verify the presence of at least one digit and one special character.

2. User Authentication Module

This module handles credential verification by comparing the entered username and password with the stored credentials. It is called only after password format validation passes.

Functions of this module:

- Compare entered username with stored username.
- Compare entered password with stored password.
- Return true if both match, false otherwise.
- Trigger Login Successful message on match.

3. Attempt Tracking Module

This module tracks the number of failed login attempts during a session and manages the remaining attempt count.

Functions of this module:

- Initialize attempts counter to 3 at session start.
- Decrement attempt counter on each failed login.
- Display remaining attempts after each failure.
- Trigger account lockout when attempts reach zero.

4. Account Lockout Module

This module monitors the attempt counter and activates lockout protection when the maximum failed attempts are reached.

Functions of this module:

- Detect when attempt count reaches zero.
- Display Account Locked due to 3 invalid attempts message.

- Terminate the login loop immediately upon lockout.
- Prevent any further login attempts for the current session.

5. User Interface and Feedback Module

This module handles all console output and user interaction, providing clear and informative messages at every stage of the login process.

Functions of this module:

- Prompt user to enter Username and Password.
- Display Invalid Password Format when format check fails.
- Display Invalid Credentials when username or password does not match.
- Display Login Successful when authentication passes.

6. Exit Module

This module handles program termination. The login loop exits when the user successfully logs in or when the account is locked after three failed attempts. The Scanner is closed cleanly before the program ends.

4. METHODOLOGY

Methodology describes the structured process followed to develop the Secure User Login and Authentication System with Password Validation. It covers all stages from understanding the security problem to designing, implementing, testing, and documenting the authentication solution.

The development process includes the following stages:

- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the security problems of existing login systems were studied and analyzed. Requirements were identified based on the need for password validation, attempt limiting, and account lockout.

Main security problems identified in existing login systems:

- Applications accepting weak passwords expose systems to password guessing attacks.
- No attempt limit allows attackers unlimited tries to guess the correct password.
- Users receive unhelpful error messages that do not indicate the reason for login failure.
- No distinction between invalid format and wrong credentials creates poor user experience.
- Absence of account lockout leaves systems vulnerable to brute-force login attacks.

Based on this analysis, the system requirements were defined to develop a secure authentication solution that enforces password validation, limits attempts, and locks accounts.

4.2. System Design

In this stage, the overall structure and architecture of the system were designed before coding began. The system was divided into independent modules, each responsible for a specific function.

Modules designed:

- Password Validation Module
- User Authentication Module
- Attempt Tracking Module
- Account Lockout Module
- User Interface and Feedback Module
- Exit Module

Class diagrams and data flow were planned to ensure a clean, maintainable code structure using object-oriented principles.

4.3. Program Development

After the design phase, the program was implemented using the Java programming language. The entire logic was written in Core Java as a console-based application with no external dependencies.

Steps followed during development:

- Creation of the isValidPassword() method with character-level validation logic.
- Implementation of the main() method with the login attempt loop using while.
- Writing conditional logic for password format check, credential check, and lockout condition.
- Handling user input through Scanner with sc.nextLine() for username and password.
- Testing of all scenarios: valid login, invalid format, wrong credentials, and lockout.

Core Java concepts applied: methods, boolean logic, character operations (Character.isUpperCase, isLowerCase, isDigit), for loop, while loop, conditional statements, and Scanner.

4.4. Testing and Validation

Testing was conducted to verify that all modules function correctly and produce the expected results. Different test cases were executed to check each feature of the system.

Test Case	Expected Result
Enter valid username and password	Login Successful is displayed
Enter valid username, wrong password	Invalid Credentials, attempts decremented
Enter password shorter than 8 chars	Invalid Password Format displayed
Enter 3 consecutive wrong attempts	Account Locked message displayed
Enter password without uppercase	Invalid Password Format displayed
Enter password without special char	Invalid Password Format displayed
Enter correct credentials on 2nd attempt	Login Successful, loop exits
Enter wrong username with valid password	Invalid Credentials, attempts decremented

This phase ensures that the system operates correctly under both normal and edge case conditions.

4.5. Result and Documentation

After successful testing, the results of the system were documented. The system accurately validates password format, checks credentials, tracks attempts, and locks accounts after three failures as expected.

Results were demonstrated through program execution and console output screenshots showing all login scenarios including successful login, invalid format, invalid credentials, and account lockout. Proper documentation was prepared to explain the design, implementation, and usage of the system.

4.6. Algorithm

Algorithm for the Secure User Login and Authentication System:

1. Start the program and initialize stored username as admin and password as Admin@123.
2. Set the attempt counter to 3.
3. Accept the user's menu choice.
4. Call isValidPassword() to validate the entered password format.
5. If format is invalid: display Invalid Password Format, decrement attempts, and check for lockout.
6. If format is valid and credentials match: display Login Successful and exit the loop.
7. If format is valid but credentials do not match: display Invalid Credentials, decrement attempts, and check for lockout.
8. If attempts reach 0: display Account Locked due to 3 invalid attempts and exit.
9. Close the Scanner and terminate the program.

4.7. Flowchart

The following flowchart describes the main program flow of the Secure User Login and Authentication System:

START
↓
User Opens Login Page
↓
Enter Username & Password
↓
Is Username Empty?
→ YES → Show Error "Enter Username" → END
→ NO ↓
Is Password Empty?
→ YES → Show Error "Enter Password" → END
→ NO ↓
Validate Password Format
(min length, special char, number, etc.)
↓
Is Password Valid Format?
→ NO → Show Error "Weak Password" → END
→ YES ↓
Check Username in Database
↓
Does User Exist?
→ NO → Show Error "User Not Found" → END
→ YES ↓
Compare Entered Password with Stored Hashed Password
↓
Is Password Correct?
→ NO → Increment Failed Attempts
↓
Attempts < Limit?
→ YES → Retry Login
→ NO → Lock Account → END
→ YES ↓
Generate Session / Token
↓
Login Successful
↓
Redirect to Dashboard
↓
END

5. SOURCE CODE

Complete Source Code:

```
import java.util.Scanner;

class UserAuthSystem {

    static String storedUsername = "admin";
    static String storedPassword = "Admin@123";

    // Password validation method
    public static boolean isValidPassword(String password) {
        if (password.length() < 8) return false;

        boolean hasUpper = false;
        boolean hasLower = false;
        boolean hasDigit = false;
        boolean hasSpecial = false;

        for (int i = 0; i < password.length(); i++) {
            char ch = password.charAt(i);

            if (Character.isUpperCase(ch)) hasUpper = true;
            else if (Character.isLowerCase(ch)) hasLower = true;
            else if (Character.isDigit(ch)) hasDigit = true;
            else hasSpecial = true;
        }

        return hasUpper && hasLower && hasDigit && hasSpecial;
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int attempts = 3;

        while (attempts > 0) {

            System.out.print("Enter Username: ");
            String username = sc.nextLine();

            System.out.print("Enter Password: ");
            String password = sc.nextLine();

            // Password format check
            if (!isValidPassword(password)) {
                attempts--;
                System.out.println("Invalid Password Format!");
            }
            // Authentication check
            else if (username.equals(storedUsername) && password.equals(storedPassword)) {
                System.out.println("Login Successful");
                break;
            }
            // Wrong credentials
            else {
                attempts--;
                System.out.println("Invalid Credentials");
            }

            // Lock condition
            if (attempts == 0) {
                System.out.println("Account Locked  due to 3 invalid attempts.");
            } else {
```

```
                System.out.println("Attempts left: " + attempts);
            }
        }

        sc.close();
    }
}
```

Code Explanation (Part by Part):

Part 1: Import & Class Declaration

```
import java.util.Scanner;
class UserAuthSystem {
```

- import java.util.Scanner -- imports the Scanner class to read the username and password entered by the user from the keyboard.
- class UserAuthSystem -- the main class containing all authentication logic. The filename must be UserAuthSystem.java to match the class name.

Part 2: Static Credential Variables

```
static String storedUsername = "admin";
static String storedPassword = "Admin@123";
```

- storedUsername -- holds the correct username (admin) that is compared during the login authentication check.
- storedPassword -- holds the correct password (Admin@123) that is compared after password format validation passes.
- static -- both variables belong to the class and can be accessed directly from static methods isValidPassword() and main() without creating an object.
-

Part 3: isValidPassword() Method

```
public static boolean isValidPassword(String password) { ... return hasUpper && hasLower
&& hasDigit && hasSpecial; }
    Scanner sc = new Scanner(System.in);
    int choice;
```

- isValidPassword(String password) -- a static boolean method that returns true only if the password meets all four format rules.
- password.length() < 8 -- immediately returns false if the password is shorter than 8 characters, without checking further.
- Character.isUpperCase(), isLowerCase(), isDigit() -- built-in Java Character class methods used inside a for loop to check each character of the password one by one.

Part 4: Main Method & Login Loop

```
Scanner sc = new Scanner(System.in);
int attempts = 3;
while (attempts > 0)
```

- Scanner → takes user input
- attempts = 3 → maximum login tries
- while (attempts > 0) → loop runs until attempts खत्म (0) or login success

Part 5: Password Format Check & Authentication Logic

```
if (!isValidPassword(password))
else if (username.equals(storedUsername) && password.equals(storedPassword))
else
```

- !isValidPassword(password) – if password format is invalid, attempts is decremented and Invalid Password Format! is printed.
- username.equals(storedUsername) && password.equals(storedPassword) – checks if both credentials match. If yes, Login Successful is printed and the loop exits via break.
- else block – runs when format is valid but credentials do not match. Prints Invalid Credentials and decrements attempts.

Part 6: Lockout Condition

```
if (attempts == 0)
sc.close();
```

- if (attempts == 0) – after decrementing, if no attempts remain the lockout message is displayed and the while loop condition becomes false, ending the program.
- Attempts left: – if attempts are still available, the remaining count is shown to inform the user.
- sc.close() – closes the Scanner resource cleanly before the program ends.

Part 7: Summary of Logic Flow

- The program first validates password format using isValidPassword(). If invalid, it skips credential checking entirely and decrements attempts.
- If format is valid, credentials are checked. A match gives Login Successful and exits. A mismatch decrements attempts and shows Invalid Credentials.

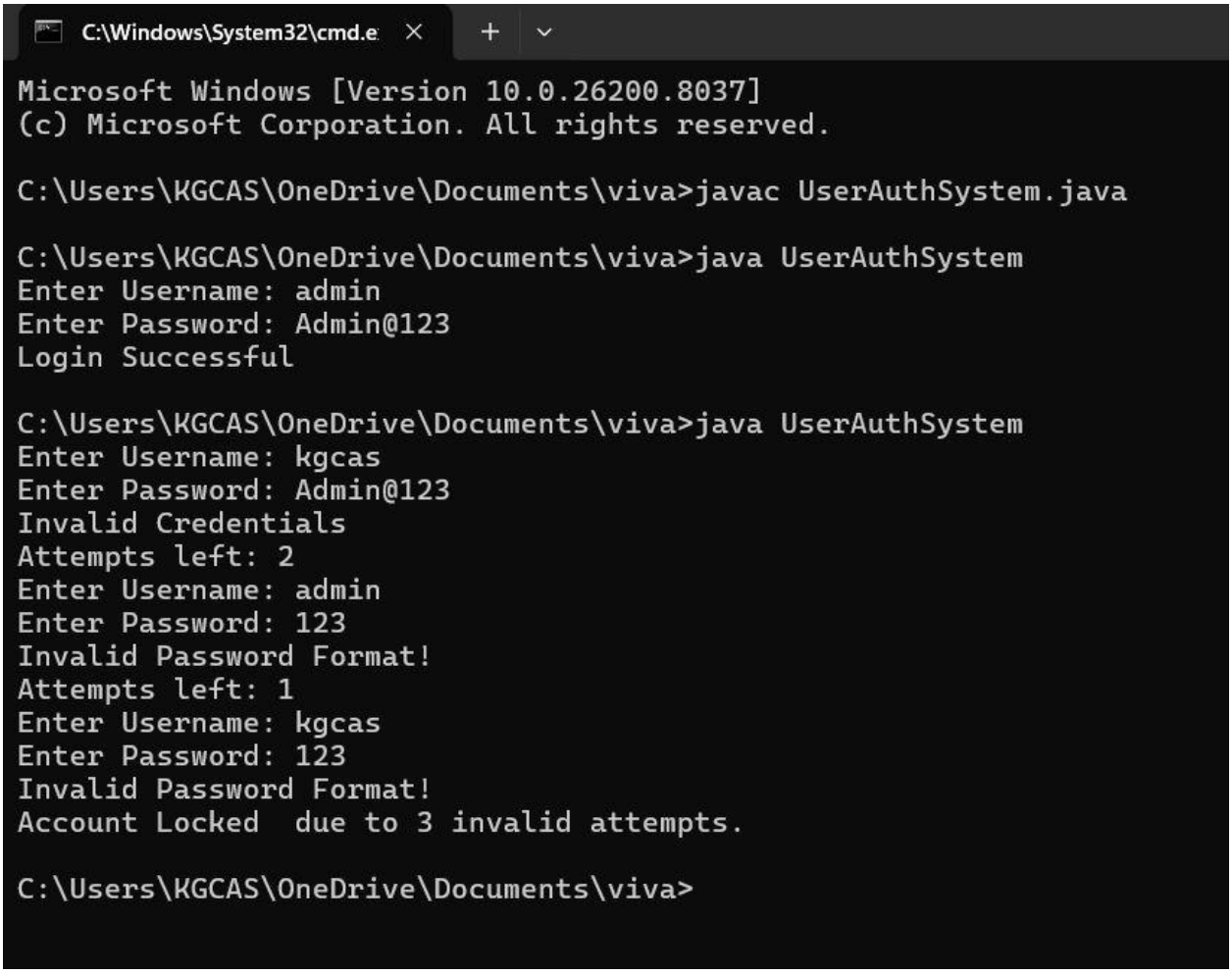
Part 8: Class and Method Structure

- UserAuthSystem contains two methods: isValidPassword() which returns a boolean, and main() which drives the login loop.
- This separation makes the code clean, readable, and easy to extend for future enhancements.

Part 9: Program Termination

- Login Successful path -- when credentials match, break exits the while loop immediately and sc.close() terminates the program cleanly.
- Lockout path -- when attempts reach 0, the while condition becomes false, the loop ends, and Account Locked message is shown before sc.close() is called.

6. OUTPUT SCREEN



Program Execution – UserAuthSystem Console Output

Figure 1: Output Screen – Program execution showing Login Successful, Invalid Credentials with attempts left, Invalid Password Format, and Account Locked after 3 invalid attempts.

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Secure User Login and Authentication System with Password Validation was developed to demonstrate secure user authentication in Java. The system successfully validates password format,

checks credentials against stored data, tracks failed login attempts, and locks the account after three consecutive failures.

During the development process, Core Java programming concepts including methods, boolean logic, character-level operations using Character class methods, while loop for attempt tracking, and conditional branching were applied. The structured design ensures clear separation between password format validation and credential authentication.

The system was tested with all scenarios including valid login, invalid password format with various missing character types, wrong credentials, and the three-attempt lockout sequence. All test cases produced the expected results, confirming that the system performs reliably and securely.

The console-based interface provides clear and descriptive feedback at every step, ensuring that users understand exactly why a login attempt failed and how many attempts remain before their account is locked.

7.2 Conclusion

The Secure User Login and Authentication System with Password Validation successfully demonstrates the practical application of Core Java programming concepts in developing a secure authentication mechanism.

The system enforces password format rules, validates user credentials, limits failed login attempts to three, and locks accounts after repeated failures. It provides specific feedback messages for each login outcome, improving both security and user experience.

This project helped in understanding how Java programming concepts such as character-level operations, boolean methods, loop control, and conditional logic can be applied to build a secure authentication system. It demonstrates how simple Java constructs can address real-world security requirements effectively.

The system can be further enhanced in future versions by adding features such as password hashing using SHA-256 or bcrypt, database-backed user management with JDBC, multi-user support, session tokens, email-based OTP verification, and persistent account lockout with admin unlock functionality.

8. REFERENCES

The following resources were referred to during the development of this project:

Books

- Herbert Schildt – Java: The Complete Reference (11th Edition, McGraw-Hill)
- Kathy Sierra & Bert Bates – Head First Java (O'Reilly Media)
- William Stallings – Cryptography and Network Security (Pearson)

Websites

- https://www.oracle.com/java – Official Java documentation
- https://www.geeksforgeeks.org – Java programming tutorials and examples
- https://owasp.org – OWASP guidelines on authentication and password security
- https://www.javatpoint.com – Java programming guides and programs
- https://docs.oracle.com/en/java – Java SE API documentation

Tools Used

- Java Development Kit (JDK) – Used to compile and run UserAuthSystem.java using javac and java commands on Windows Command Prompt.